OTUS

Kotlin Backend Developer.Professional

otus.ru

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Практические принципы работы с БД

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
TG OTUS Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Программа курса

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Выбор проекта	Маркетинг аналитика	Security	Архитек-тура	DevOps	
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде	Обзор экосист.	
Модуль 9: Проектная работа	Защита					

Окатов Сергей
автор программы, руководитель курса

- кандидат физ.-мат. наук
- 15 лет научной работы в университетах РФ, Европы, США
- 10 лет комм. разработки в стартапах и корпорациях (Сбер, Ланит, Солар) в качестве руководителя разработки, архитектора
- Специализация
  - Котлин
  - Большие данные
  - Распределенные системы
  - Архитектура

План полета

- Данные и Базы данных
- Требования к Базам Данных
- Процесс разработки
- Проектирование хранилищ
- Рефлексия

Цели вебинара

После занятия вы:

1. Перечислить современные виды БД и их особенности
2. Сформулировать бизнес-требования к БД
3. Подготовить инфраструктуру БД для разработки и тестирования с применением БД

Смысл

Зачем вам это уметь

1. На рынке присутствует большое количество БД, необходимо понимать какие тенденции присутствуют
2. Решаемые задачи бывают разные и под разные задачи выбираются разные БД
3. Правильно подготовленная инфраструктура значительно сокращает количество проблем при выводе в пром.эксплуатацию.

Домашнее задание

1. Выполнить InMemory реализацию репозитария
2. Сделать автоматические тесты для репозитория
3. Создать персистентное хранилище для выпускного проекта
4. Реализовать миграцию для персистентного хранилища
5. Интегрировать хранилище в выпускной проект

Виды баз данных

Типы данных

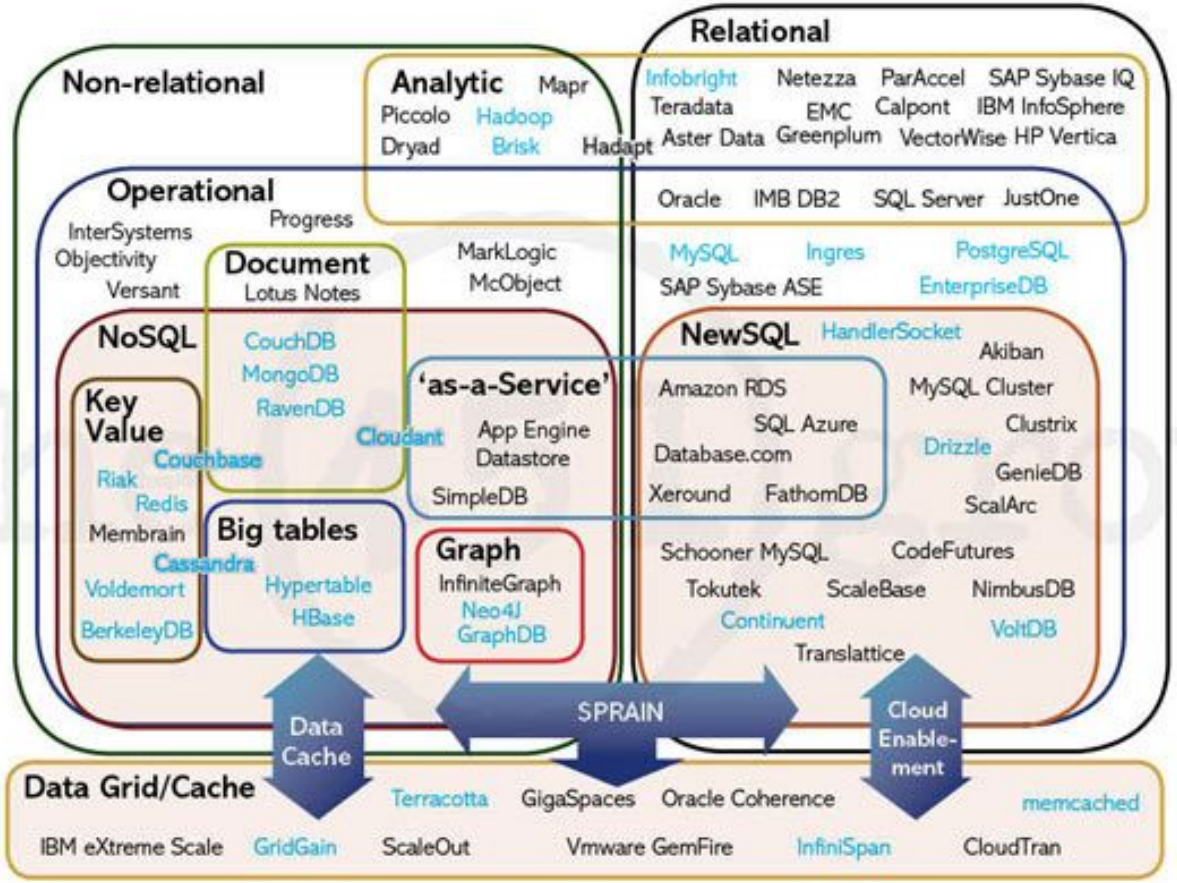
Категория	Типы данных	Требования к хранению
Важность	Критичные Второстепенные	Репликация Резервное копирование
Частота	Горячие Тёплые Холодные	SSD/in-memory Гибридные HDD/архивы
Размер	Массивные Средние Малые	Распределённые Традиционные СХД Локальные
Структура	Структурированные Полуструктур. Неструктур.	Реляционные БД NoSQL Объектные
Срок	Временные Краткосрочные Среднесроч. Постоянные	Кэширование Оперативные Архивные Долгосрочные
Прочее	Конфиденциальные Распределенные Транзакционные Потоковые	Шифрование Географическое OLTP/OLAP Streaming

Данные очень разные, далеко не всегда получается их все обслужить одной БД

Сколько баз данных существует?

Грубая оценка:

1. Известные, популярные: 50-70
2. Нишевые: 150-250
3. Малоизвестные, экспериментальные, заброшенные: 500+



Обобщенные виды баз данных

Название вида	Год появления	Особенности	Назначение	Список ярких представителей
SQL (Реляционные)	1970-е	Табличная структура, строгая схема, SQL, ACID-транзакции.	Финансы, ERP, CRM, транзакционные системы.	Oracle, MySQL, PostgreSQL, SQL Server
NoSQL	2000-е	Гибкая схема, горизонтальная масштабируемость, отказ от SQL.	Неструктурированные данные, высоконагруженные системы.	MongoDB, Cassandra, Redis, Couchbase
NewSQL	2010-е	ACID-транзакции + горизонтальная масштабируемость.	Высоконагруженные транзакционные системы.	CockroachDB, Google Spanner, TiDB, VoltDB
Многомодельные	2010-е	Поддержка нескольких моделей данных в одной СУБД.	Гибридные проекты, микросервисы.	ArangoDB, Azure Cosmos DB, Couchbase, OrientDB

Старые виды баз данных

Название вида	Год появления	Особенности	Назначение	Список ярких представителей
Сетевые	1960-е	Модель CODASYL, гибкие связи между записями.	Телеком, логистика, унаследованные системы.	IDMS, TurboIMAGE, Raima DB
Иерархические	1960-е	Древовидная структура, родительско-дочерние связи.	Устаревшие системы, файловые структуры, XML.	IBM IMS, Windows Registry
Ключ-значение	1990-е	Простая модель (ключ + значение), высокая скорость.	Кэширование, сессии, очереди.	Redis, DynamoDB, Memcached, etcd
In-Memory	1990-е	Данные в оперативной памяти, минимальные задержки.	Торговые системы, игровые сервисы, кэширование.	Redis, SAP HANA, Apache Ignite, VoltDB
Объектно-ориентированные	1990-е	Хранение объектов с наследованием и полиморфизмом.	CAD, мультимедиа, сложные структуры.	ObjectDB, Versant, db4o
Пространственные (Spatial)	1990-е	Поддержка геоданных, индексы для пространственных запросов.	ГИС, картография, логистика, навигация.	PostGIS, Oracle Spatial, MongoDB (Geospatial)

Современные виды БД

Название вида	Год появления	Особенности	Назначение	Список ярких представителей
Документные	2000-е	JSON/XML/BSON-документы, вложенные структуры.	Каталоги, CMS, API-ориентированные приложения.	MongoDB, CouchDB, Firebase, Elasticsearch
Столбцовые	2000-е	Оптимизация для чтения по колонкам, сжатие данных.	Аналитика, Big Data, метрики.	Cassandra, HBase, Amazon Redshift, ClickHouse
Графовые	2000-е	Узлы, рёбра, свойства, алгоритмы обхода графов.	Социальные сети, рекомендации, фрод-детекция.	Neo4j, Amazon Neptune, ArangoDB, TigerGraph
Распределенные	2000-е	Шардирование, репликация, согласованность в распределённой среде.	Глобальные приложения, высокая доступность.	Cassandra, CockroachDB, Google Spanner, YugabyteDB
Поисковые	2000-е	Индексация текста, полнотекстовый поиск, ранжирование.	Поисковые движки, аналитика логов, рекомендации.	Elasticsearch, Apache Solr, OpenSearch
Временные ряды	2010-е	Оптимизация под временные метки, сжатие временных данных.	IoT, мониторинг, телеметрия, финансы.	InfluxDB, TimescaleDB, Prometheus, OpenTSDB

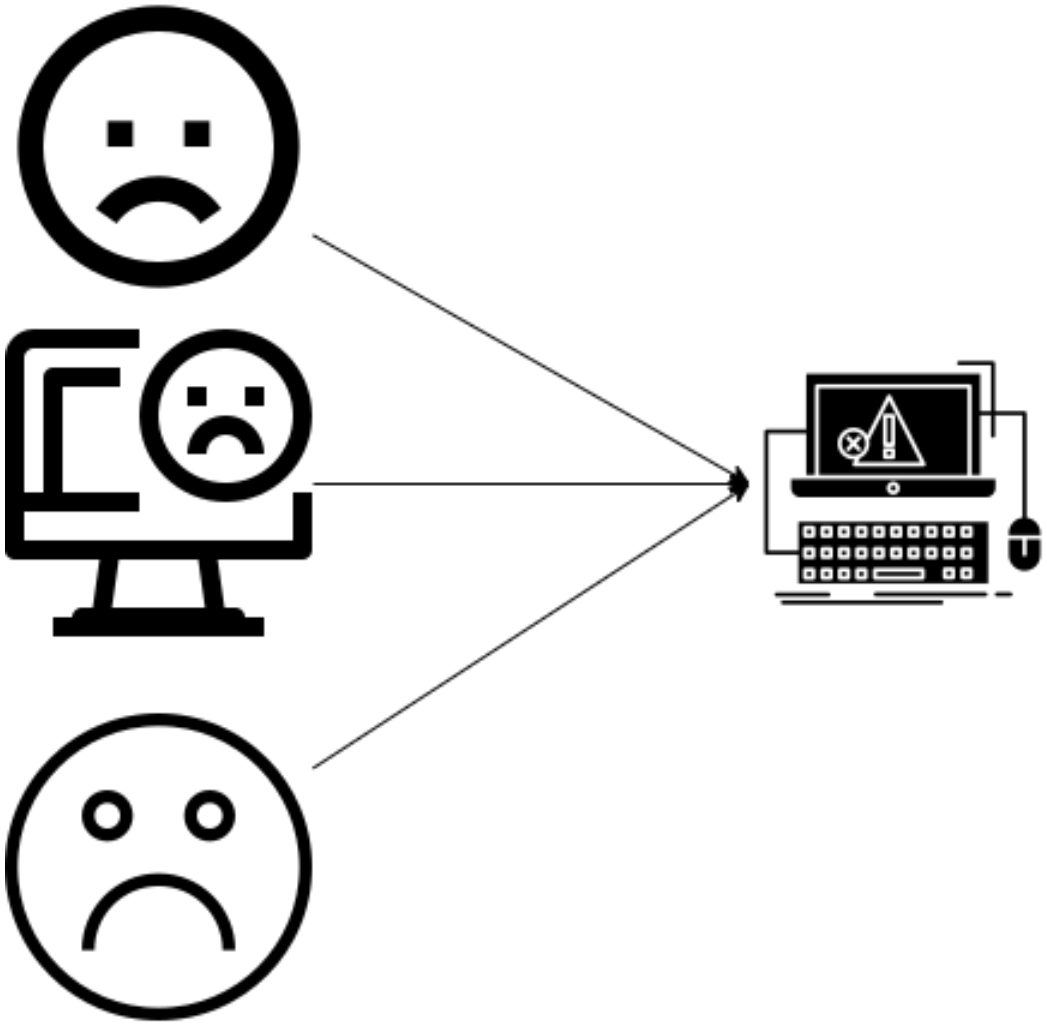
Последние тенденции: поверх S3

1. Cloud Native подход
2. Специальные форматы для S3-хранилищ
3. Специальные хранилища метаданных (HiveMetastore, HMS)
4. SQL поверх облачных форматов:
   a. Trino
   b. StarRocks
   c. Doris
   d. RisingWave
5. Большие данные, распределенное хранение, отказоустойчивость, OLAP

Feature	Apache Hudi	Apache Iceberg	Delta Lake
Primary Use Case	Fast data updates, streaming, and ETL	Large-scale analytics, schema evolution	ACID transactions, data quality, Spark integration
Data Update	Supports upserts and deletes	Supports row-level updates	Supports updates and deletes
Performance	Optimized for low-latency updates	Optimized for large-scale analytics	Optimized for Spark workloads
Partitioning	User-defined partitioning	Hidden partitioning (auto)	User-defined partitioning
Schema Evolution	Supports schema evolution	Supports complex schema evolution	Supports schema evolution with constraints
Time Travel	Limited, with incremental queries	Advanced time travel capabilities	Strong time travel capabilities
Integration	Works well with Spark, Hive, Flink	Broad support, including Spark, Presto, Flink	Best with Spark, growing ecosystem
Community	Growing, with commercial support	Backed by major cloud platforms	Strong support from Databricks and the Spark community

Бизнес-требования к персистентному хранилищу

У бизнеса есть требования к ИС и БД



Deploying!

Why not working?

Требования к БД

Требование	Варианты удовлетворения
Производительность	OLTP (PostgreSQL, MySQL) / OLAP (ClickHouse, Redshift) / HTAP (TiDB, CockroachDB) / Кэширование (Redis)
Масштабируемость	Горизонтальное (Cassandra, CockroachDB) / Вертикальное (Oracle) / Шардирование (MongoDB)
Доступность	Репликация (синхронная/асинхронная), кластеризация, RTO < 1 мин (Aurora, MongoDB Atlas)
Безопасность	Шифрование данных (TLS, AES), RBAC, аудит (GDPR/HIPAA), защита от инъекций
Надежность данных	ACID-гарантии (PostgreSQL) / Eventual Consistency (Cassandra) / Резервное копирование
Стоимость	Open-source (PostgreSQL) / Облачные managed-решения (RDS, Cosmos DB) / Лицензии (Oracle)
Соответствие	GDPR (ПДн), PCI DSS (плат.карты), HIPAA (мед.) совместимость (AWS Aurora, Snowflake)
Экосистема	Интеграция с ETL (Airflow), BI-инструментами (Power BI), драйверы (Python/Java/.NET)

Как выбрать БД

Практический чек-лист для выбора

1. **Требования проекта**: OLTP или OLAP? Нужны ли сложные JOIN-запросы?
   a. OnLine Transaction Processing - много транзакций
   b. OnLine Analytical Processing - сложный анализ
   c. Hybrid Transactional Analytical Processing - универсальные
2. **Объем данных**: Рост данных через 1–3 года.
   a. Обычные данные - PostgreSQL
   b. Большие данные - распределенные БД
3. **Нагрузка**: Пиковые значения RPS (запросов в секунду).
   a. Обычная нагрузка - PostgreSQL
   b. Нагруженная система - InMemory, Распределенные БД
4. **Команда**: Опыт работы с конкретной СУБД.
5. **Бюджет**: Лицензии, облачные расходы, поддержка.
6. **Экосистема**: Интеграция с текущим стеком технологий.

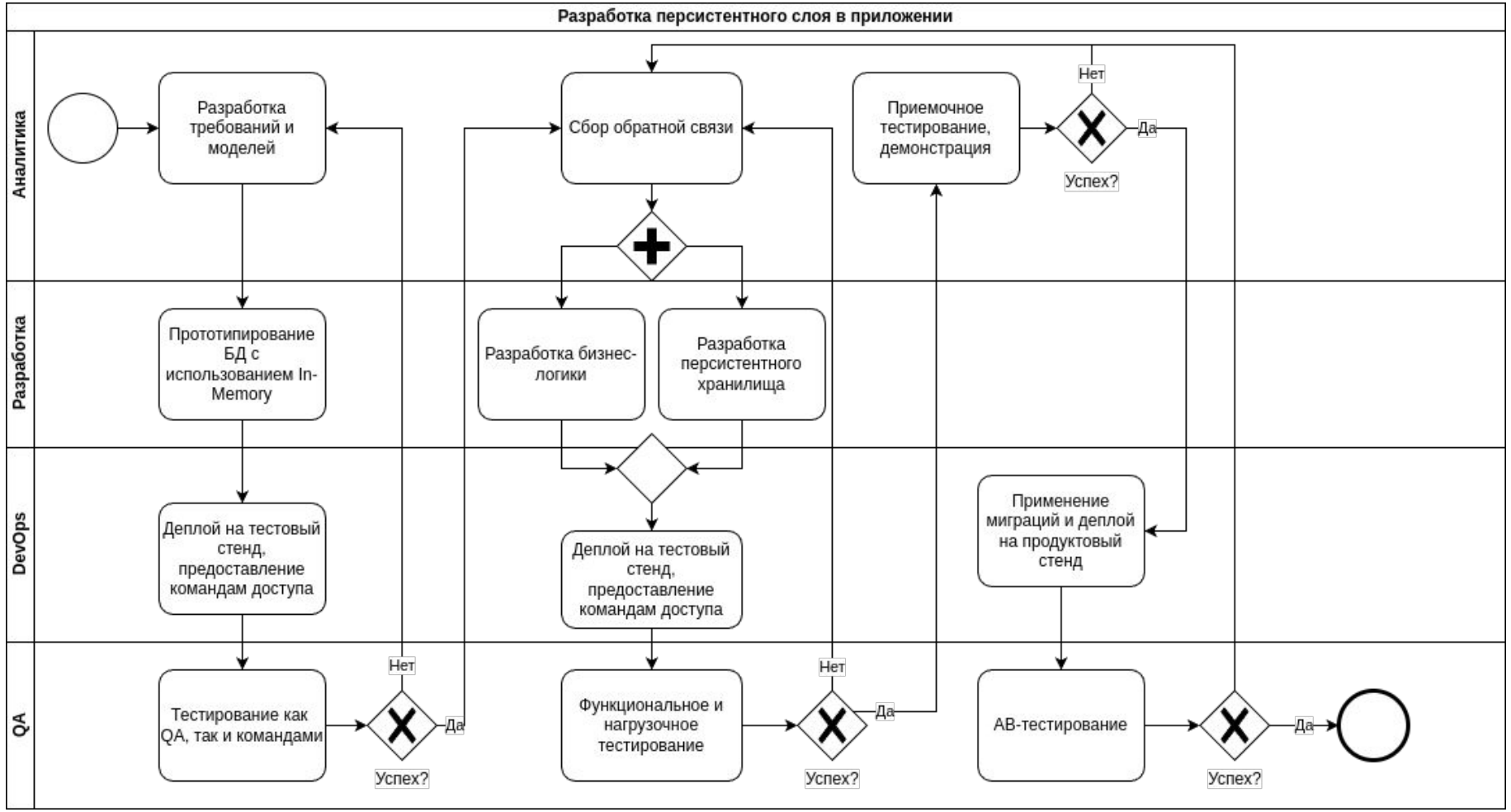
Процесс разработки слоя персистентного хранения в приложении

Процесс разработки персист.слоя

Этапы:

1. In-Memory
2. Бизнес-логика
3. Персистентное хранилище

Причины…



Процесс разработки персист.слоя

Этапы:

1. In-Memory
2. Бизнес-логика
3. Персистентное хранилище

Причины:

1. InMemory примитивная, сделаем быстро, сможем вывести на стенд для дальнейшей интеграции.
2. Боевая БД требует долгой разработки схемы и оптимизаций
3. Автотесты в InMemory проходят на порядки быстрее, что упрощает разработку и поддержку проекта
4. Даже в боевом режиме можем тестировать с помощью InMemory
5. При тестировании InMemory выявляются дополнительные нюансы работы с хранилищем, поэтому проработка финальной версии будет более качественной и быстрой

Процесс разработки персист.слоя

Слом шаблона:

1. Появились подходы и проекты с переносом бизнес-логики и интегарций в БД

Инструменты:

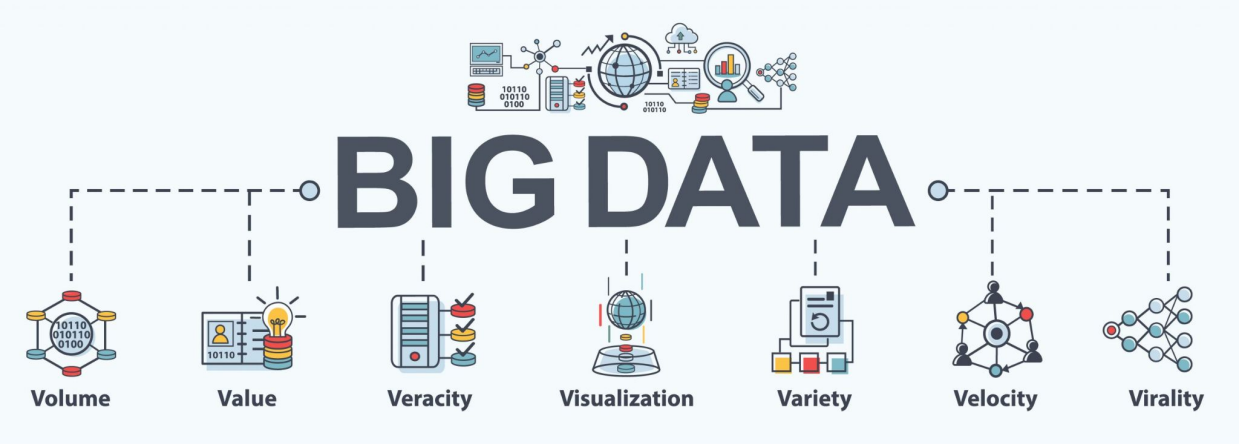
1. Apache DBT - построение аналитических запросов в SQL
2. Spark SQL, Flink SQL, DataFusion, Arroyo - процессоры данных с поддержкой SQL
3. RisingWave - потоковая БД

Рабочий сценарий в проекте

Используем большие данные

Последствия

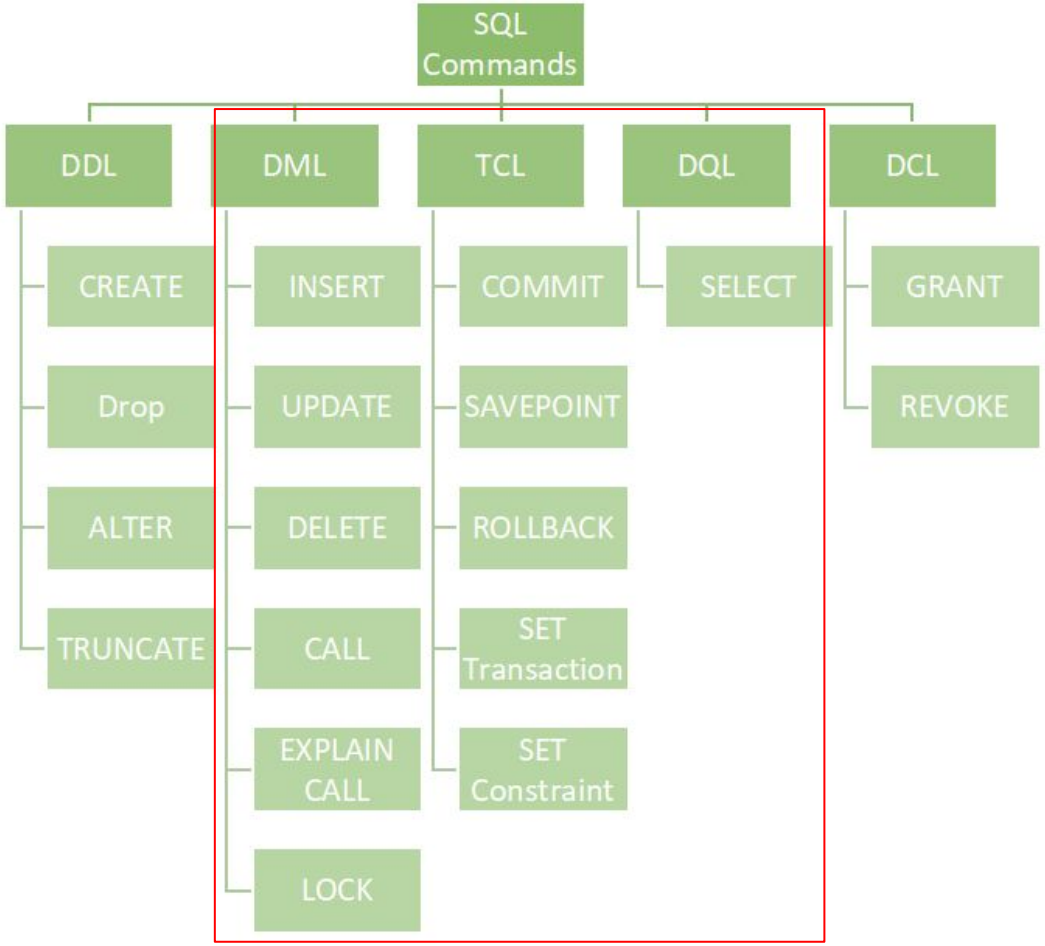
1. Слишком большое время восстановления (Downtime) по сравнению с RPO (recovery point objective)
2. Слишком большое время миграции схемы
3. Слишком большое время выполнения запроса
4. Слишком большие требования к RAM и HDD/SDD



Учитываем безопасность

Последствия

1. DDL - Data Definition Layer
2. DML - Data Manipulation Layer
3. TCL - Transaction Control Layer
4. DQL - Data Query Layer
5. DCL - Data Control Layer



<u>В runtime доступны только DML, DQL и TCL</u>

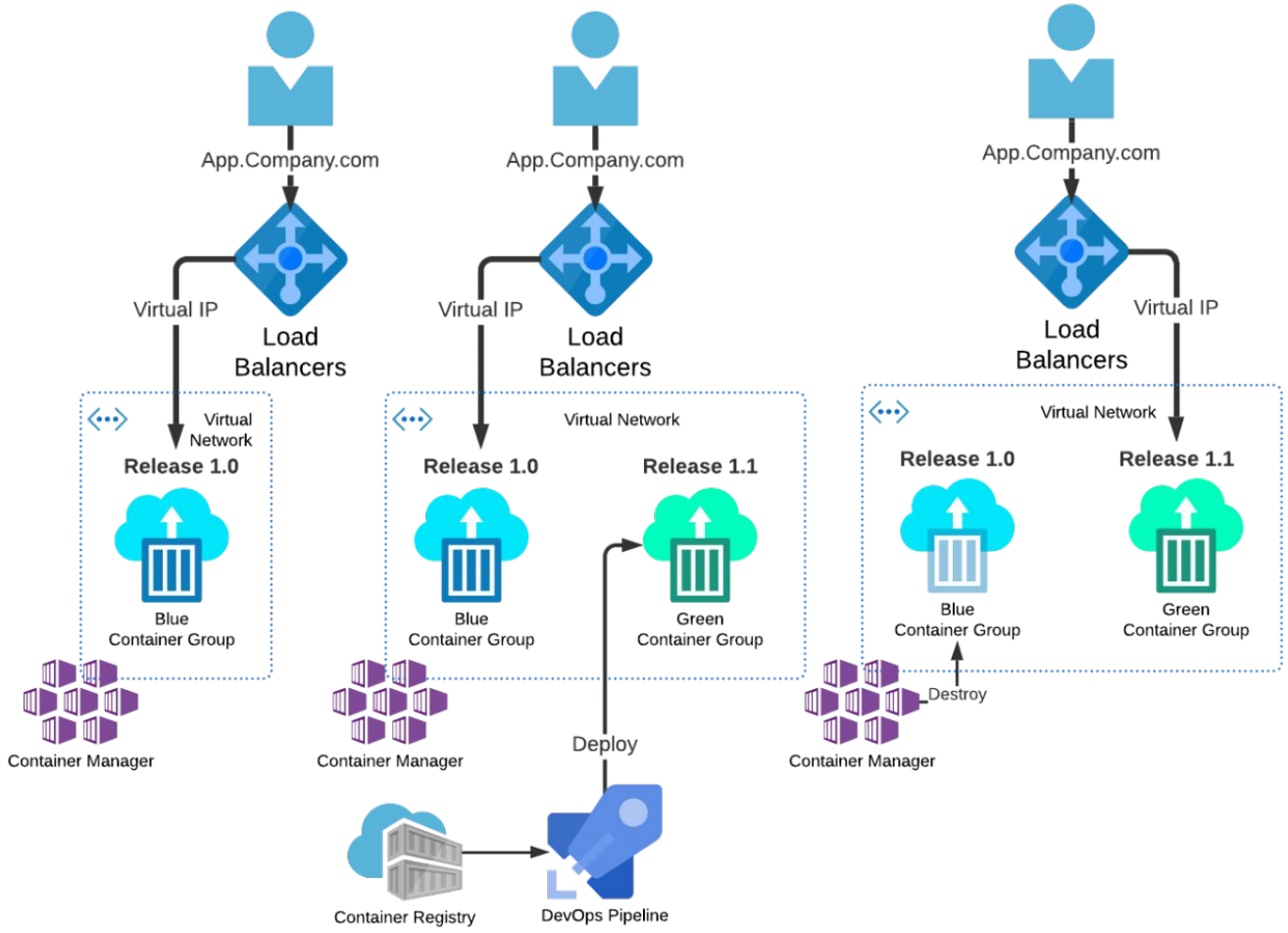
<u>В startup доступн DDL</u>

<u>DCL только для админов</u>

Требуем Zero Downtime Deployment

Blue-Green Deployment

1. Старая версия - Blue
2. Новавя версия - Green
3. Переключение мгновенное
4. Проблемы:
   - Как обеспечить консистентность данных между Blue и Green?

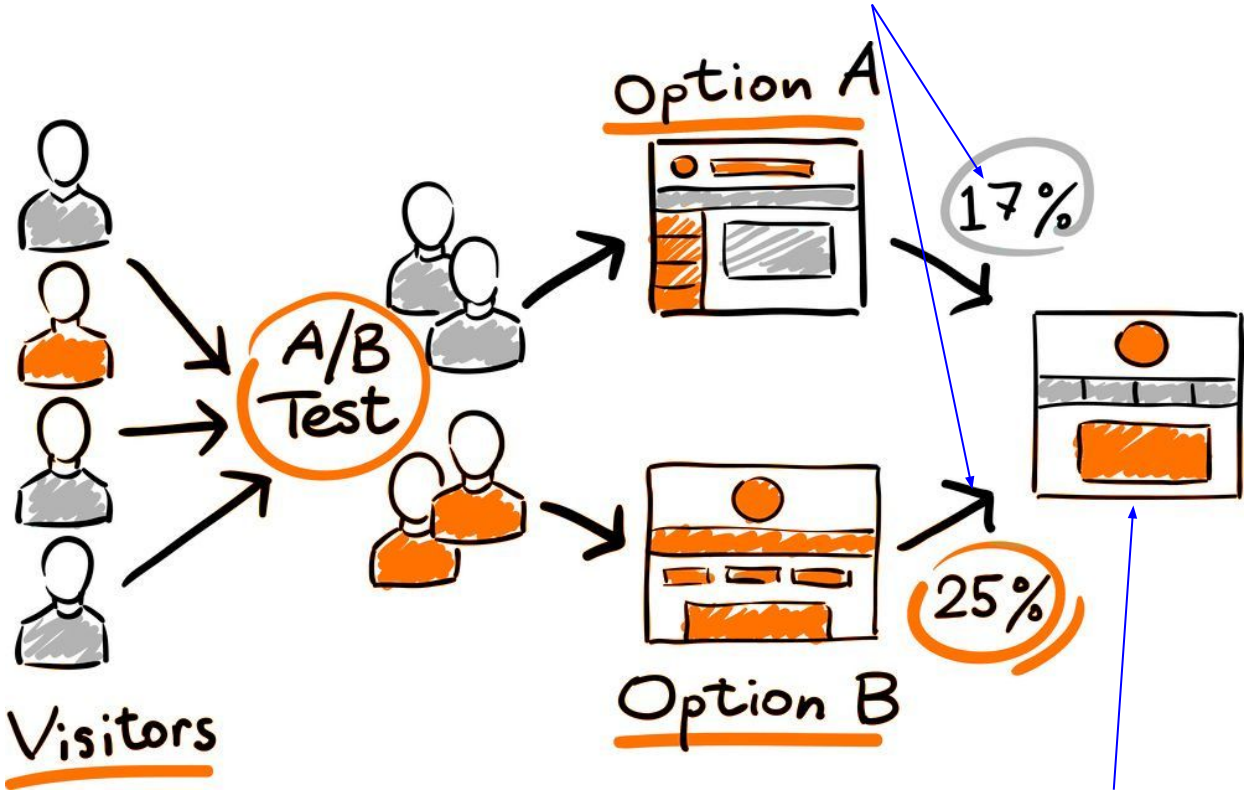


Планируем A/B Testing

Плавная/условная миграция на новую версию, HADI-цикл:

1. Option A - реперная реализация
2. Option B - новая версия
3. Работа A и B одновременная
4. Проблемы:
   - Как обеспечить консистентность данных между A и B при отличиях в структурах БД?

Метрики эффективности
(напр., конверсия)



Анализ

Принципы разработки БД

Принципы:

1. Data First - данные первичны, код вторичен
2. БД должна быть отделена от кода
3. У БД должен быть свой релизный цикл, своя нумерация версий
4. Обновление БД должно выполняться отдельным миграционным процессом, не завязанным на другие компоненты (приложение-пользователь)
5. Взаимодействие с клиентами через контракт (view)

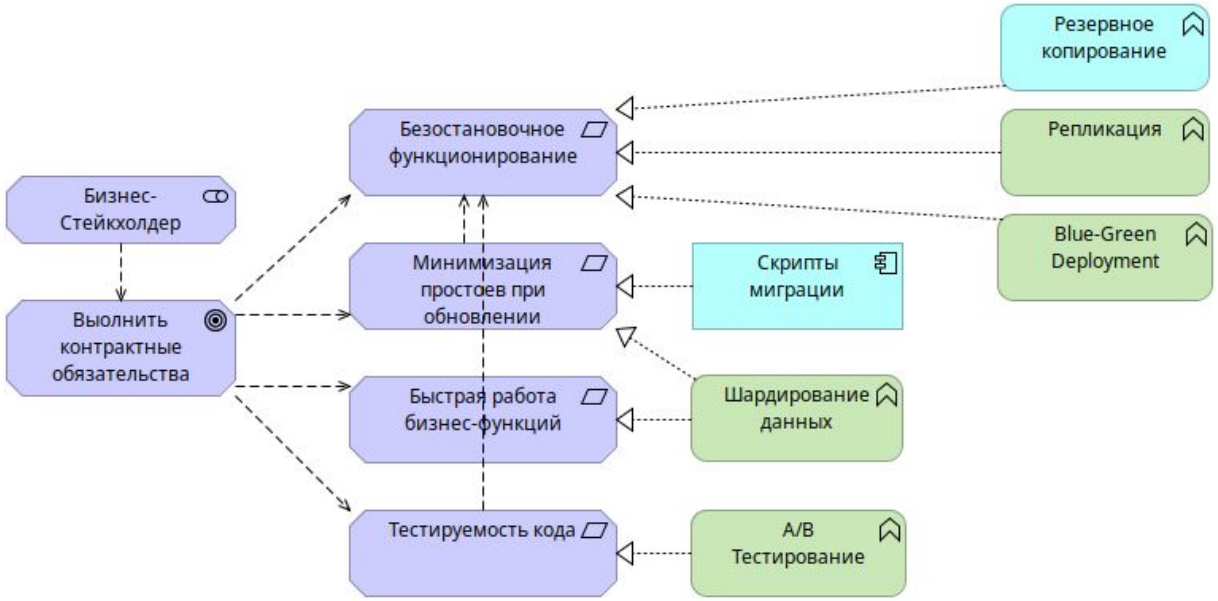
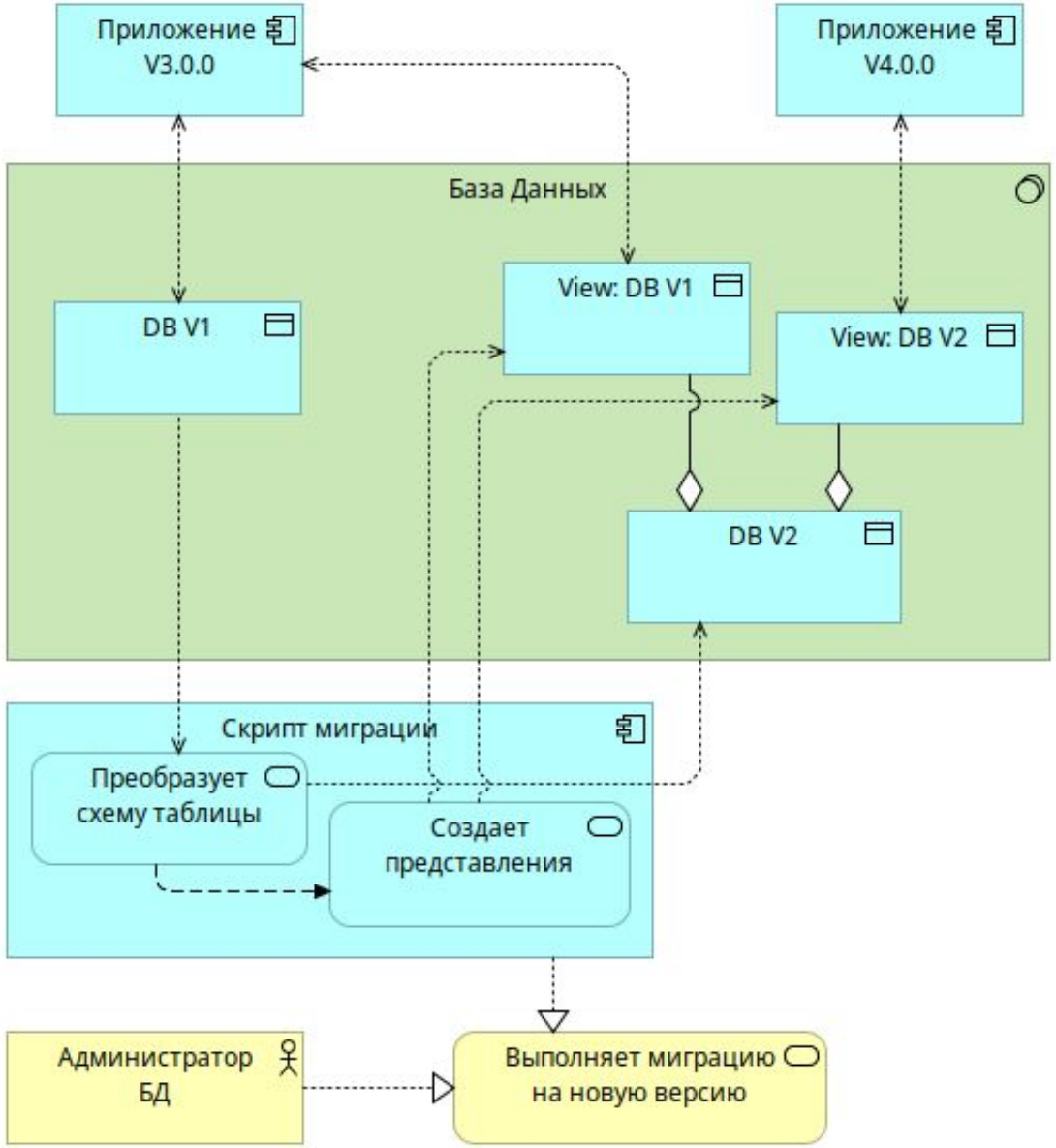


Схема миграции БД

Принципы:

1. Data First - данные первичны, код вторичен
2. БД должна быть отделена от кода
3. У БД должен быть свой релизный цикл, своя нумерация версий
4. Обновление БД должно выполняться отдельным миграционным процессом, не завязанным на другие компоненты (приложение-пользователь)
5. Взаимодействие с клиентами через контракт (view)



Инструменты миграций БД

Bytebase

Название	Лицензия	Язык разработки	Языки описания миграций	Поддерживаемые языки разработки	Поддерживаемые БД (группы)
Liquibase	Apache 2.0	Java	XML, YAML, JSON, SQL	Java, .NET, Python, Groovy, **CLI**	**61+ СУБД** (SQL: PostgreSQL, MySQL, Oracle и др.; NoSQL: MongoDB, Cassandra, Scylla; Graph: Neo4j, ArcadeDB, OrientDB; Timeseries: InfluxDB, TimescaleDB)
Flyway	Apache 2.0	Java	SQL, Java	Java, .NET, Go, Node.js, Python, **CLI**	**28 SQL СУБД**: PostgreSQL, MySQL, Oracle, SQL Server, Snowflake, Redshift и др.
Alembic	MIT	Python	Python, SQL	Python, **CLI**	SQL (6+ через SQLAlchemy: PostgreSQL, MySQL, SQLite и др.)
Django Migrations	BSD	Python	Python	Python, **CLI**	SQL (4: PostgreSQL, MySQL, SQLite, Oracle)
Sqitch	MIT	Perl	SQL, DSL	Perl, **CLI**	SQL (10+), включая PostgreSQL, MySQL, SQLite
TypeORM Migrations	MIT	TypeScript	TypeScript, SQL	TypeScript/JavaScript, **CLI**	SQL (6+), NoSQL (MongoDB), Graph (через расширения)
Evolve	MIT	C#	SQL	C#, .NET, **CLI**	SQL (7+: PostgreSQL, MySQL, SQL Server и др.)
dbmate	MIT	Go	SQL	Go, **CLI**	SQL (10+: PostgreSQL, MySQL, SQLite и др.)
Phinx	MIT	PHP	PHP, SQL	PHP, **CLI**	SQL (6+: MySQL, PostgreSQL, SQLite и др.)

LIVE

Реализация миграции

Рефлексия

Тезисы

Подведем итоги

1. Баз данных очень много, основые типы - SQL, NoSQL, NewSQL, многомодельные
2. Основные требования к хранилищу с: Производительность, Масштабируемость, Доступность, Безопасность, Надежность данных, Стоимость, Соответствие, Экосистема
3. Процесс разработки: InMemory, конечное хранилище для ускорения первой версии, для быстрых тестов, для более тщательной проработки финальной версии
4. funcs
5. Библиотеки: Hibernate, Spring Data, Exposed, JOOQ

Домашнее задание

1. Выполнить InMemory реализацию репозитария
2. Сделать автоматические тесты для репозитория
3. Создать персистентное хранилище для выпускного проекта
4. Реализовать миграцию для персистентного хранилища
5. Интегрировать хранилище в выпускной проект

Срок: 17.10.2025

Рефлексия

С какими впечатлениями уходите с вебинара?

Как будете применять на практике то,
что узнали на вебинаре?

Цели вебинара

Проверка достижения целей:

1. Перечислить современные виды БД и их особенности
2. Сформулировать бизнес-требования к БД
3. Подготовить инфраструктуру БД для разработки и тестирования с применением БД

Список материалов для изучения

1. https://github.com/otuskotlin/202505-ok-marketplace
2. Liquibase: https://www.liquibase.com/
3. Бэнчмарки разных БД: https://benchmark.clickhouse.com/
4. Ранжировка БД: https://db-engines.com/en/ranking
5. Уровни управления БД: https://www.geeksforgeeks.org/sql-ddl-dql-dml-dcl-tcl-commands/
6. Критерии выбора СУБД: https://swcats.kz/blog/kak-vybrat-subd-dlya-novogo-proekta
7. Green Blue Deployment: https://habr.com/ru/companies/abdigital/articles/668478/
8. A/B Тестирование: https://habr.com/ru/articles/233911/

Следующий вебинар

30.09.2025

Модуль 7: Работа с базами данных

Создание репозитория

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары